# PROYECTO FINAL - SIMULADOR DE BATALLAS POKEMON

Maestría en Inteligencia Artificial, Universidad Sergio Arboleda
Asignatura: Computacion Cientifica 2
Docente: Guillermo Andres De Mendoza Corrales
Fecha de entrega: Noviembre 27 de 2026

---

## 1. Objetivos

El proyecto consiste en desarrollar un simulador de batallas Pokémon de 3 contra 3 por turnos para dos jugadores (Jugador 1 vs Jugador 2) compartiendo la misma terminal. El programa solicitará secuencialmente las acciones de cada jugador, calculará las prioridades según el modelo físico/mecanico del juego y procesará las fórmulas matemáticas oficiales para determinar el resultado del turno.
**Programación Orientada a Objetos (POO):** Aplicar abstracción, encapsulamiento, herencia y polimorfismo para modelar las entidades del juego.

**Modelado de sistemas y estados:** Gestionar correctamente la máquina de estados de un combate por turnos (cambios de estado de salud, turnos, prioridad de acciones).

**Computación Estocástica:** Implementar modelos de probabilidad para determinar precisión, golpes críticos y dispersión de daño en cada acción.

**Persistencia de datos:** Diseñar e implementar una base de datos relacional SQLite que almacene el catálogo de Pokémon, movimientos, la tabla de efectividad de tipos y el historial de batallas, desacoplando los datos de la lógica del programa.

## 2. Descripcion

### ¿Qué es un Pokémon?

En el contexto de la simulación, un Pokémon es una entidad de combate definida por un conjunto de atributos numéricos (estadísticas) y un perfil elemental (tipos). Cada Pokémon posee un estado de salud ($HP$) y cinco estadísticas principales ($Atk$, $Def$, $Sp.Atk$, $Sp.Def$, $Speed$). Su desempeño en batalla depende directamente de la relación entre su tipo elemental y el tipo de los movimientos que ejecuta o recibe.

Pueden buscar cada pokemon y sus estadísticas en el siguiewnte link: https://pokemondb.net/pokedex/dragonite



### datos de la Pokédex

| | |
|---|---|
| Nacional n.º | **0149** |
| Tipo | DRAGÓN VUELO |
| Especies | Pokémon dragón |
| Altura | 2,2 m (7′03″) |
| Peso | 210,0 kg (463,0 libras) |
| Talento | 1. Enfoque interno<br>multiescala (habilidad oculta) |
| | 0149 (Rojo/Azul/Amarillo)<br>0243 (Oro/Plata/Cristal)<br>0149 (Rojo Fuego/Verde Hoja)<br>0248 (Oro Corazón/Plata Alma)<br>0147 (X/Y — Montaña Kalos) |

### Capacitación

| | |
|---|---|
| rendimiento de los vehículos eléctricos | 3 Ataque |
| Tasa de captura | 45 (5,9% con Poké Ball, PS al máximo) |
| Amistad básica | 35 (inferior a lo normal) |
| Exp. base. | 300 |
| Índice de crecimiento | Lento |

### Cría

## Estadísticas base

| | | | Min | Máximo |
|---|---|---|---|---|
| HP | 91 | | 292 | 386 |
| Ataque | 134 | | 245 | 403 |
| Defensa | 95 | | 175 | 317 |
| Ataque especial | 100 | | 184 | 328 |
| Defensa especial | 100 | | 184 | 328 |
| Velocidad | 80 | | 148 | 284 |
| Total | **600** | | | |

Los rangos que se muestran a la derecha corresponden a un Pokémon de nivel 100. Los valores máximos se basan en una naturaleza beneficiosa (252 EVs, 31 IVs); los valores mínimos se basan en una naturaleza perjudicial (0 EVs, 0 IVs).

Las estadísticas de un pokémon se encuentran entre el valor mínimo y máximo. Para efectos de este proyecto podrán tomar cualquier valor.

### Movimientos

#### Movimientos aprendidos al subir de nivel

*Dragonite* aprende los siguientes movimientos en Pokémon Legends: ZA en los niveles especificados.

| Nivel. | Mover | Tipo | Gato. | Fuerza | Ampl. |
|---|---|---|---|---|---|
| 1 | Corte aéreo | VUELO | | 75 | 95 |
| 1 | Mirada lasciva | NORMAL | | — | 100 |
| 1 | Abordar | NORMAL | | 40 | 100 |
| 5 | Tornado | DRAGÓN | | 40 | 100 |
| 10 | Onda de trueno | ELÉCTRICO | | — | 90 |
| 15 | Derribar | NORMAL | | 90 | 85 |
| 20 | Deslizamiento de ruptura | DRAGÓN | | 60 | 100 |
| 25 | Brutal Swing | OSCURO | | 60 | 100 |
| 30 | Dragon Rush | DRAGÓN | | 100 | 75 |
| 40 | Salvaguardia | NORMAL | | — | — |
| 50 | Velocidad extrema | NORMAL | | 80 | 100 |
| 55 | Escándalo | DRAGÓN | | 120 | 100 |
| 60 | Rayo hiperrápido | NORMAL | | 150 | 90 |

#### Movimientos aprendidos durante la evolución

*Dragonite* aprende los siguientes movimientos cuando evoluciona en Pokémon Legends: ZA (independientemente del nivel).

| Mover | Tipo | Gato. | Fuerza | Ampl. |
|---|---|---|---|---|
| Huracán | VUELO | | 110 | 70 |

#### Movimientos aprendidos mediante recordatorio

*Dragonite* aprende los siguientes movimientos adicionales al usar la función 'Recordar

#### Movimientos aprendidos por TM

*Dragonite* es compatible con estas Máquinas Técnicas en Pokémon Legends: ZA:

| TM | Mover | Tipo | Gato. | Fuerza | Ampl. |
|---|---|---|---|---|---|
| 02 | Garra de Dragón | DRAGÓN | | 80 | 100 |
| 05 | Rugido | NORMAL | | — | — |
| 08 | Onda de trueno | ELÉCTRICO | | — | 90 |
| 10 | Rompe ladrillos | LUCHA | | 75 | 100 |
| 12 | Deslizamiento de rocas | ROCA | | 75 | 90 |
| 13 | Rayo de hielo | HIELO | | 90 | 100 |
| 16 | Pantalla de luz | PSÍQUICO | | — | — |
| 17 | Proteger | NORMAL | | — | — |
| 22 | As aéreo | VUELO | | 60 | ∞ |
| 23 | Puñetazo de trueno | ELÉCTRICO | | 75 | 100 |
| 24 | Ponche de hielo | HIELO | | 75 | 100 |
| 27 | Rápido | NORMAL | | 60 | ∞ |
| 29 | Puñetazo de fuego | FUEGO | | 75 | 100 |
| 35 | Perdurar | NORMAL | | — | — |
| 36 | Tumba de roca | ROCA | | 60 | 95 |
| 38 | Explosión de fuego | FUEGO | | 110 | 85 |
| 41 | Pulso de agua | AGUA | | 60 | 100 |
| 43 | Volar | VUELO | | 90 | 95 |
| 44 | Rayo hiperrápido | NORMAL | | 150 | 90 |
| 47 | Agilidad | PSÍQUICO | | — | — |
| 51 | Salvaguardia | NORMAL | | — | — |

Ejemplo de un movimiento: [https://pokemondb.net/move/earthquake](https://pokemondb.net/move/earthquake)

# Terremoto (movimiento)

Contenido | Texto descriptivo | Aprendido subiendo de nivel | Aprendimos sobre la evolución. | Aprendido por TM

## Mover datos

| | |
|---|---|
| Tipo | SUELO |
| Categoría | Físico |
| Fuerza | 100 |
| Exactitud | 100 |
| PÁGINAS | 10 (máx. 16) |
| ¿Establece contacto? | No |
| Introducido | Generación 1 |

## Efectos

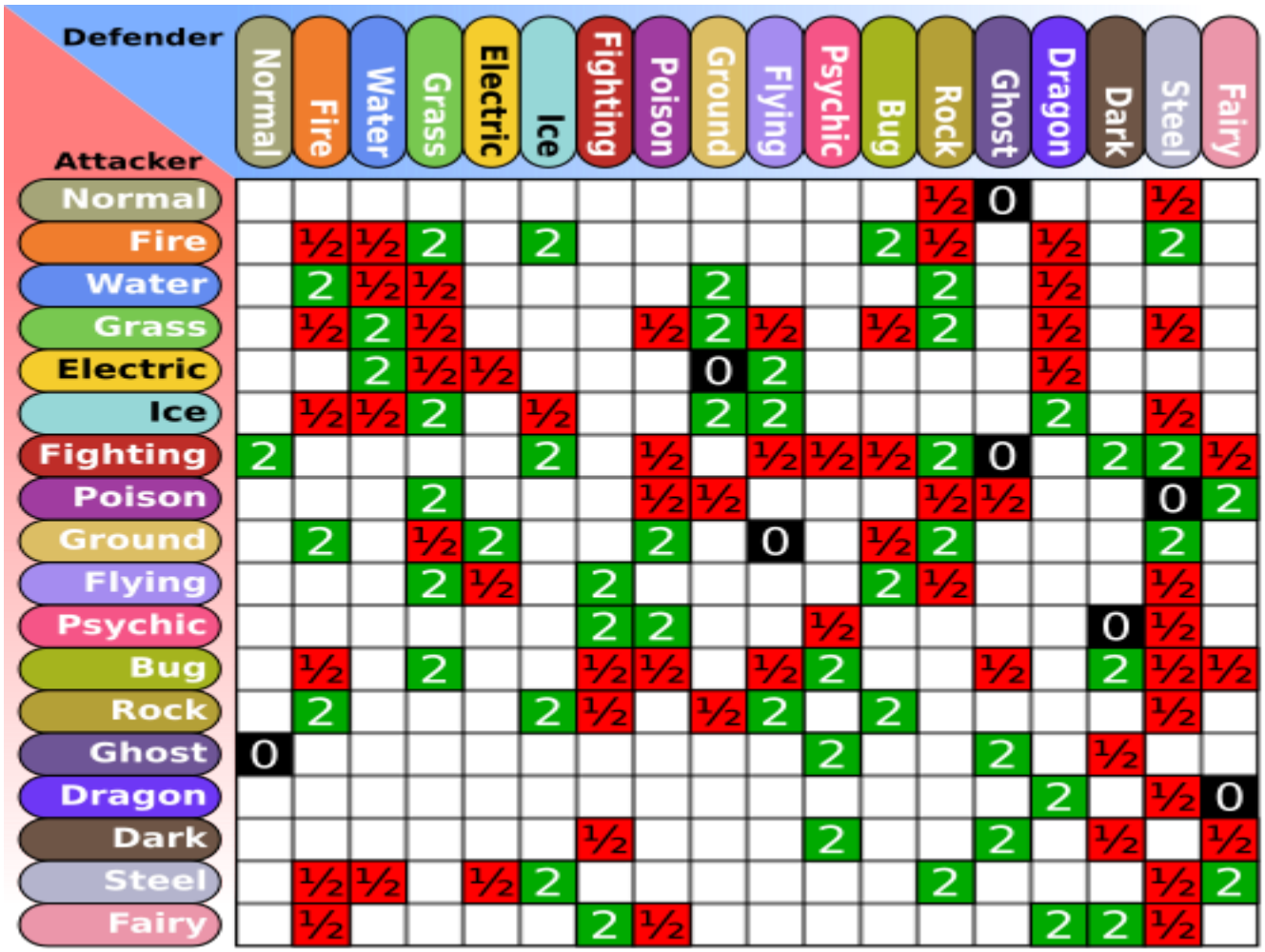
*Terremoto* inflige daño y golpeará con el doble de potencia si el oponente está bajo tierra debido al movimiento Excavar .

### Efectos de movimiento Z

Cuando un Pokémon lleva Groundium Z y usa su Poder Z, *Terremoto* se convierte en Furia Tectónica y tiene una potencia base **de 180** .

### Mecánica de Tipos y Efectividad

Cada Pokémon y cada movimiento pertenecen a uno o dos tipos elementales. La interacción entre tipos determina un multiplicador de daño ($Efectividad$) en la fórmula de combate:



- Súper Efectivo ($\times\ 2.0$ o $\times\ 4.0$): El ataque inflige el doble de daño (o cuádruple si el objetivo tiene doble tipo vulnerable).
- Daño Normal ($\times\ 1.0$): El ataque inflige el daño estándar.
- Poco Efectivo ($\times\ 0.5$ o $\times\ 0.25$): El ataque inflige la mitad de daño (o la cuarta parte).
- Inmune ($\times\ 0.0$): El ataque no inflige daño.

## 3. Requerimientos para el pool de pokemones

Para garantizar la variedad estratégica y la correcta prueba de la matriz de tipos en el proyecto, el sistema debe tener configurados un **pool mínimo de 20 Pokémon predefinidos**, almacenados en una **base de datos SQLite** (ver sección 4). Ningún dato del pool debe quedar escrito directamente en el código fuente.

**3.1 Cobertura Total de Tipos:** Entre los 20 Pokémon de la base de datos, deben estar representados los **18 tipos oficiales** de la franquicia (Fuego, Agua, Planta, Eléctrico, Hielo, Lucha, Veneno, Tierra, Volador, Psíquico, Bicho, Roca, Fantasma, Dragón, Acero, Siniestro, Hada y Normal).

**3.2 Atributos Completos:** Cada registro de la tabla pokemon debe incluir obligatoriamente:

- `id`: Identificador único.
- `nombre`: Nombre del Pokémon.
- `tipo_1` y `tipo_2`: Referencia (FK) a la tabla tipos (ej. "Fuego", "Volador" o null si es monotipo).
- `estadisticas`: Objetos o valores numéricos para $HP$, $Atk$, $Def$, $Sp.Atk$, $Sp.Def$ y $Speed$.
- `movimientos`: 4 movimientos asignados, relacionados a través de la tabla intermedia pokemon_movimientos.

## 4. Persistencia en base de datos (SQLite)

Toda la información del juego debe residir en un archivo de base de datos SQLite (por ejemplo `pokemon.db`), gestionado con el módulo estándar `sqlite3` de Python (no requiere instalar nada adicional). Al iniciar, el programa debe consultar la base de datos para listar los Pokémon disponibles y construir los objetos `Pokemon` y `Movimiento` a partir de los registros obtenidos. La tabla de efectividad de tipos también debe leerse de la base de datos, no estar codificada como un diccionario en el código.

### 4.1 Modelo de datos (esquema mínimo)

El esquema debe estar normalizado y contener, como mínimo, las siguientes tablas:

- **`tipos`**: `id` (PK), `nombre` (único). Debe contener los 18 tipos oficiales.
- **`efectividad_tipos`**: `tipo_atacante_id` (FK → tipos), `tipo_defensor_id` (FK → tipos), `multiplicador` (0, 0.5, 1 o 2). Clave primaria compuesta por ambos tipos; debe contener las 18 × 18 = 324 combinaciones.
- **`movimientos`**: `id` (PK), `nombre` (único), `tipo_id` (FK → tipos), `categoria` (Físico o Especial), `potencia`, `precision` (0-100), `pp_max`.
- **`pokemon`**: `id` (PK), `nombre` (único), `tipo_1_id` (FK → tipos, obligatorio), `tipo_2_id` (FK → tipos, NULL si es monotipo), `hp`, `ataque`, `defensa`, `at_especial`, `def_especial`, `velocidad`.
- **`pokemon_movimientos`**: `pokemon_id` (FK → pokemon), `movimiento_id` (FK → movimientos). Relación muchos a muchos; cada Pokémon debe tener exactamente 4 movimientos.
- **`batallas`**: `id` (PK), `fecha`, `jugador_1`, `jugador_2`, `ganador`, `numero_turnos`. Historial de combates finalizados.

### 4.2 Requerimientos de implementación

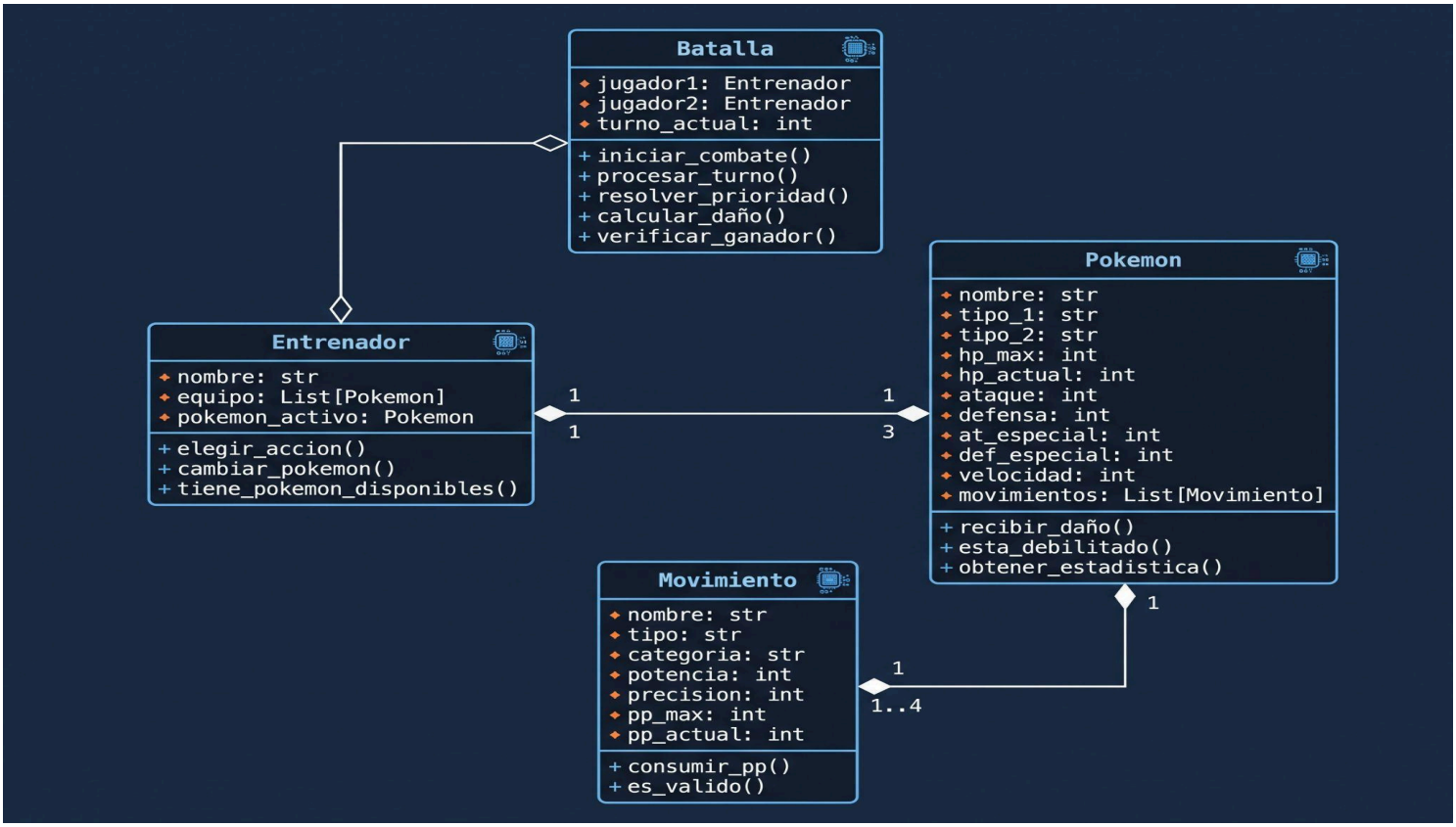
- **Script de creación y carga:** Entregar un script (por ejemplo `crear_bd.py`, o `schema.sql` + `datos.sql`) que cree todas las tablas desde cero y cargue los tipos, la matriz de efectividad, los movimientos y los 20 Pokémon. Ejecutarlo dos veces no debe duplicar datos.

- **Integridad referencial:** Definir claves primarias, claves foráneas, restricciones `NOT NULL`/`UNIQUE`/`CHECK` donde aplique (por ejemplo `precision BETWEEN 0 AND 100`) y activar `PRAGMA foreign_keys = ON` en cada conexión.
- **Consultas parametrizadas:** Todas las consultas deben usar parámetros (`?`) en lugar de concatenar cadenas, para evitar inyección SQL.
- **Capa de acceso a datos separada:** Las clases del dominio (`Pokemon`, `Movimiento`, `Entrenador`, `Batalla`) no deben ejecutar SQL directamente; toda interacción con la base de datos se concentra en la clase `RepositorioPokemon` (ver sección 5).
- **Estado de batalla en memoria:** La base de datos almacena los valores base (`hp`, `pp_max`). Los valores que cambian durante el combate (`hp_actual`, `pp_actual`) viven sólo en los objetos en memoria y no deben modificar el catálogo.
- **Historial:** Al terminar cada combate se debe insertar un registro en la tabla `batallas`, y el menú principal debe ofrecer una opción para consultar el historial (por ejemplo, últimas batallas y número de victorias por jugador usando `GROUP BY`).

### 4.3 Entregables asociados

- Archivo `pokemon.db` ya poblado y el script que permite regenerarlo.
- Diagrama entidad-relación (E-R) del esquema, incluido junto al diagrama UML de clases.

## 5. Clases propuestas




**Clase `Movimiento`:**

- Atributos: `nombre`, `tipo`, `categoria` (*Físico* o *Especial*), `potencia`, `precision` (0-100), `pp_max`, `pp_actual`.
- Métodos: `consumir_pp()`, `es_valido()`.

**Clase `Pokemon`:**

- Atributos: `nombre`, `tipo_1`, `tipo_2` (opcional), `hp_max`, `hp_actual`, `ataque`, `defensa`, `at_especial`, `def_especial`, `velocidad`, `movimientos` (lista/array de hasta 4 instancias de `Movimiento`).
- Métodos: `recibir_daño(cantidad)`, `esta_debilitado()`, `atacar(movimiento, objetivo)`, `obtener_estadistica()`.

**Clase `Entrenador` (o `Jugador`):**

- Atributos: `nombre`, `equipo` (lista de 3 `Pokemon`), `pokemon_activo`.
- Métodos: `elegir_accion()`, `cambiar_pokemon(nuevo_indice)`, `tiene_pokemon_disponibles()`.

**Clase `Batalla` (Motor principal):**

- Atributos: `jugador1`, `jugador2`, `turno_actual`.
- Métodos: `iniciar_combate()`, `procesar_turno(accion_p1, accion_p2)`, `resolver_prioridad()`, `calcular_daño()`, `verificar_ganador()`.

**Clase `RepositorioPokemon` (Acceso a datos):**

- Atributos: `ruta_bd`, `conexion`.
- Métodos: `listar_pokemon()`, `cargar_pokemon(id)` (retorna un objeto `Pokemon` con sus 4 instancias de `Movimiento`), `obtener_efectividad(tipo_ataque, tipo_defensor)`, `registrar_batalla(batalla)`, `obtener_historial()`, `cerrar()`.

## 6. Mecanica de combate por turnos

#### Fase de Comandos:

En la terminal, se presentan las opciones al Jugador 1 (ocultando o limpiando pantalla opcionalmente para mantener privacidad) y luego al Jugador 2.

- Opciones por turno: **[1] Atacar** (seleccionar movimiento) o **[2] Cambiar Pokémon** (seleccionar reserva vivo).

#### Prioridad de Ejecución:

- El cambio de Pokémon tiene prioridad absoluta sobre cualquier ataque.
- Si ambos eligen cambiar, se ejecutan en orden aleatorio o según la velocidad de sus Pokémon activos.
- Si ambos eligen atacar, ataca primero el Pokémon con mayor estadística de velocidad. En caso de empate, la prioridad se decide mediante un sorteo aleatorio de probabilidad $p = 0.5$.

#### Cálculo Numérico del Daño:

$$\text{Daño} = \left( \frac{\left(\frac{2 \times \text{Nivel}}{5} + 2\right) \times \text{Potencia} \times \frac{\text{Ataque}}{\text{Defensa}}}{50} + 2 \right) \times \text{Modificador}$$

*(Asumir Nivel = 50 para todos los Pokémon).*

- **Reglas para Ataque / Defensa:**
  - Si el movimiento es **Físico**, se usa Ataque del atacante y Defensa del objetivo.

    - Si el movimiento es **Especial**, se usa At_Especial del atacante y Def_Especial del objetivo.
- **Modificador:**

$$Modificador = STAB \times Efectividad \times Crítico \times Aleatorio$$

- **STAB (Same Type Attack Bonus):** $1.5$ si el tipo del movimiento coincide con algún tipo del Pokémon atacante; $1.0$ si no.
- **Efectividad:** Multiplicador obtenido de la tabla de tipos $(0.0, 0.5, 1.0, 2.0, 4.0)$.
- **Crítico:** $1.5$ si ocurre un golpe crítico; $1.0$ en caso contrario.
- **Aleatorio:** Valor continuo uniformemente distribuido $U(0.85, 1.00)$.

Cuando un Pokémon tiene dos tipos (por ejemplo, **Gyarados**, que es de tipo **Agua / Volador**), la efectividad total del ataque se calcula **multiplicando individualmente** la efectividad del tipo del ataque contra cada uno de los dos tipos del Pokémon defensor.

La fórmula es:

$$EfectividadTotal = Efectividad(TipoAtaque \rightarrow Tipo1) \times Efectividad(TipoAtaque \rightarrow Tipo2)$$

#### Simulación Estocástica

- **Precisión:** Un ataque acierta solo si un valor aleatorio $r \sim U(1, 100) \leq Precisión$.
- **Probabilidad de Crítico:** Cada ataque exitoso tiene un \$6.25\%\$ de probabilidad $(1/16)$ de asestar un golpe crítico.

#### Interfaz en Terminal (CLI)

- Presentación clara del estado actual al inicio de cada turno:
    - Nombres de los Pokémon activos y sus barras/porcentajes de HP actual vs. HP máximo.
    - Puntos de Poder (PP) restantes para cada movimiento al seleccionar un ataque.

#### Reglas adicionales

Cuando un Pokémon cae a $0HP$: El jugador que perdió al Pokémon debe seleccionar un reemplazo **inmediatamente** antes de continuar

## 7. Ejemplos de interacciones (Casos de prueba)

### Caso 1: Validación de Prioridad por Velocidad y Ventaja de Tipo Básica

- **Propósito:** Validar que el Pokémon más rápido ataca primero y que el modificador de tipo súper efectivo ($\times\ 2.0$) con STAB ($\times\ 1.5$) se aplique correctamente.
- **Atacante:** *Jolteon* (Eléctrico) — **Velocidad: 130**

    1. Ataque usado: **Rayo** (Especial, Tipo: Eléctrico, Potencia: 90, Precisión: 100%)
    2. Estadística $Sp.Atk$: 110
- **Defensor:** *Gyarados* (Agua / Volador) — **Velocidad: 81**

    1. Estadística $Sp.Def$: 100
- **Cálculo Esperado:**
    1. **Orden:** Jolteon actúa primero por tener mayor velocidad $(130 > 81)$.
    2. **Categoría:** Especial → Usa $Sp.Atk$ de Jolteon (110) y $Sp.Def$ de Gyarados (100).
    3. **STAB:** $1.5$ (Rayo es Eléctrico y Jolteon es Eléctrico).

4. **Efectividad:** Eléctrico es súper efectivo contra Agua ($\times\ 2.0$) y Volador ($\times\ 2.0$) → Multiplicador total de tipo $= 4.0$.
5. **Daño final:**

$$DañoBase = \left(\frac{22 \times 90 \times \frac{110}{100}}{50} + 2\right) = \left(\frac{2178}{50} + 2\right) = 45.56 \rightarrow 45$$

6. $DañoTotal = 45 \times 1.5(STAB) \times 4.0(Efectividad) = 270\ puntosdedaño.$

## Caso 2: Validación de Ataque Físico vs Especial e Inmunidad

- **Propósito:** Comprobar la diferencia entre estadísticas físicas/especiales y el manejo estricto del multiplicador de inmunidad ($\times\ 0.0$).
- **Atacante:** *Snorlax* (Normal) — **Velocidad: 30**
    1. Ataque usado: **Terremoto** (Físico, Tipo: Tierra, Potencia: 100, Precisión: 100%)
    2. Estadística $Atk$: 110
- **Defensor:** *Charizard* (Fuego / Volador) — **Velocidad: 100**

    1. Estadística $Def$: 78
- **Cálculo Esperado:**
    1. **Orden:** Charizard actúa primero por velocidad ($100 > 30$).
    2. **Efectividad:** Los ataques de tipo Tierra no afectan a los tipos Volador ($Efectividad = 0.0$).
    3. **Daño final:** $0$ puntos de daño (Mensaje en consola: *"No afecta a Charizard..."*).

## Caso 3: Validación de Golpe Crítico y Resistencia (Poco Efectivo)

- **Propósito:** Verificar el cálculo cuando ocurre un golpe crítico (probabilidad del \$6.25\%\$, multiplicador $\times\ 1.5$) combinado con resistencia tipo ($\times\ 0.5$).
- **Atacante:** *Charizard* (Fuego / Volador)
    1. Ataque usado: **Lanzallamas** (Especial, Tipo: Fuego, Potencia: 90, Precisión: 100%)
    2. Estadística $Sp.Atk$: 129
- **Defensor:** *Blastoise* (Agua)
    1. Estadística $Sp.Def$: 105
- **Cálculo Esperado:**
    1. **STAB:** $1.5$ (Lanzallamas es Fuego y Charizard es Fuego).
    2. **Efectividad:** Fuego es poco efectivo contra Agua ($\times\ 0.5$).
    3. **Crítico:** Si el motor genera $r \leq 0.0625$, se aplica multiplicador por Crítico $= 1.5$.
    4. **Daño final:**

$$DañoBase = \left(\frac{22 \times 90 \times \frac{129}{105}}{50} + 2\right) = \left(\frac{2432.57}{50} + 2\right) = 50.65 \rightarrow 50$$

$$DañoTotal = 50 \times 1.5(STAB) \times 0.5(Efectividad) \times 1.5(Crítico) = 56.25 \rightarrow 56puntos.$$

## Caso 4: Validación de Prioridad por Cambio de Pokémon y Fallo por Precisión

- **Propósito:** Confirmar que la acción "Cambiar Pokémon" interrumpe el orden por velocidad y validar el fallo de un movimiento por precisión.
- **Jugador 1:** Elige la acción **Cambiar Pokémon** (cambia a *Gengar*).
- **Jugador 2:** Elige la acción **Atacar** con *Machamp* usando **Puño Dinámico** (Físico, Tipo: Lucha, Potencia: 100, **Precisión: 50%**).
- **Cálculo Esperado:**

1. **Prioridad:** El cambio de Pokémon del Jugador 1 se procesa **antes** de que el Jugador 2 ejecute su ataque, sin importar las velocidades.
2. **Verificación de Precisión:** El sistema genera un número aleatorio $r \sim U(1, 100)$.
    - Si $r > 50$, el ataque falla automáticamente.
    - No se deduce $HP$ del objetivo y se muestra el mensaje: *"¡El ataque de Machamp ha fallado!"*.
3. **Consumo de PP:** Los Puntos de Poder ($PP$) del movimiento **Puño Dinámico** deben reducirse en 1 unidad aunque el ataque haya fallado.

## 8. Criterio de evaluación

| Criterio | Descripcion | Valor |
|---|---|---|
| Diseño y Estructura POO | Correcto modelado de clases, responsabilidades bien delimitadas, encapsulamiento y UML. | 10% |
| Lógica de Combate y Reglas | Correcta implementación de prioridades de velocidad, cambios de Pokémon, debilidad/resistencia y flujo de turnos | 25% |
| Modelo Matemático y Estocástico | Precisión en la fórmula de daño, cálculo de STAB, generación de aleatorios para precisión/críticos. | 25% |
| Persistencia en SQLite | Esquema normalizado con diagrama E-R, script de creación y carga, integridad referencial, consultas parametrizadas, capa de acceso a datos separada y registro/consulta del historial de batallas. | 30% |
| Interfaz CLI | Manejo de errores de entrada del usuario, flujo amigable. Pueden hacer esta capa apoyándose con IA | 10% |